

Hypothesis Testing with E-Values

Chapter 12: Using E-Values to Combine Dependent P-Values

Based on Ramdas & Wang

Where we are

- **Chapters 8–11** developed *e-merging*: combining several e-values $E_1, \dots, E_K$ into one e-value. The punchline there: this is *easy* under arbitrary dependence – e.g. the plain average $\frac{1}{K} \sum_k E_k$ is already valid, by linearity of expectation alone, no matter how the E_k depend on each other.
- **Chapter 12** asks the analogous question for *p-values*: given $P_1, \dots, P_K$ that are p-values for the same or related nulls, possibly with completely unknown/adversarial dependence, how do we combine them into a single valid p-value?
- Surprise: this is *much harder* for p-values than for e-values – and the clean way to solve it is to *convert p-values to e-values, merge those, and convert back*.
- Roadmap:
 - 12.1: the arithmetic mean of p-values, and why you must multiply it by 2;
 - 12.2: general merging functions for p-values under arbitrary dependence, and why *all* of them secretly come from e-values;
 - 12.3: what changes if the p-values are *exchangeable*;
 - 12.4: what changes if you are willing to *randomize*.

Notation you'll need I

Setup (recap from Ch. 8, 11)

Fix an atomless null probability measure $\mathbb{P}$ (results extend to a general null class $\mathcal{P}$) and an integer $K \geq 2$. We consider K *p*-variables $P_1, \dots, P_K$ for $\mathbb{P}$, i.e. each satisfies $\mathbb{P}(P_k \leq t) \leq t$ for all $t \in [0, 1]$, and K *e*-variables $E_1, \dots, E_K$, i.e. each satisfies $\mathbb{E}^{\mathbb{P}}[E_k] \leq 1$. Write $\mathbf{p} = (p_1, \dots, p_K)$, $\mathcal{K} = \{1, \dots, K\}$, and let $\mathfrak{U}$ be the set of all *p*-variables for $\mathbb{P}$.

“Arbitrary dependence” – what this phrase actually means

Nothing at all is assumed about the *joint* law of $(P_1, \dots, P_K)$ (equivalently, of $(E_1, \dots, E_K)$) beyond each coordinate being marginally valid on its own. We must build procedures that work under the *worst-case* dependence structure (copula) – including cases where the P_k are deterministic functions of one another. This is exactly the same “arbitrary dependence” regime used for e-merging in Chapters 8 and 11; the difference is that *p*-values behave much worse than *e*-values under it.

Notation you'll need II

P-merging function (Definition 12.9)

A *p-merging function* of K p-values is an increasing Borel map $F : [0, \infty)^K \rightarrow [0, \infty)$ such that $F(P_1, \dots, P_K)$ is itself a p-variable *for every choice* of p-variables $P_1, \dots, P_K$ – i.e. valid under arbitrary dependence among them. F is *symmetric* if permuting the inputs doesn't change the output, *homogeneous* if $F(\lambda \mathbf{p}) = \lambda F(\mathbf{p})$ for $\lambda \in (0, 1]$ (with $F(\mathbf{p}) \leq 1$), and *admissible* if no other p-merging function is uniformly at least as small (i.e. it cannot be improved).

Plain English: a p-merging function is a formula that eats K p-values and outputs a single number that is guaranteed to be a valid p-value, **no matter how** the K inputs are statistically entangled.

Notation you'll need III

Calibrators (recap from Ch. 2)

A *calibrator* is a decreasing function $f : [0, \infty) \rightarrow [0, \infty]$ with $\int_0^1 f(p) \, dp \leq 1$ and $f \equiv 0$ on $(1, \infty)$. For any p-variable P , $f(P)$ is an e-variable: $\mathbb{E}^{\mathbb{P}}[f(P)] \leq \int_0^1 f(p) \, dp \leq 1$. Calibrators are exactly the “p-value $\rightarrow$ e-value” converters we will lean on throughout this chapter. f is *convex* if it is a convex function on $[0, 1]$ – this extra structure is what makes *averaging* p-values (rather than just e-values) work out, as we will see in 12.1.

Notation you'll need IV

Exchangeability – explained plainly

$(P_1, \dots, P_K)$ is *exchangeable* if, for every permutation π of $\{1, \dots, K\}$, $(P_{\pi(1)}, \dots, P_{\pi(K)})$ has the *same joint distribution* as $(P_1, \dots, P_K)$.

Analogy: imagine drawing K cards from a shuffled deck and writing down their values *in the order you drew them*. The values can be highly dependent (e.g. correlated because they came from the same deck) – but relabeling/permuting the order in which you recorded them does not change anything statistically, because the deck was shuffled. Exchangeability is a much weaker, more common assumption than independence: e.g. p-values from repeated random sample-splits of the *same* dataset are exchangeable (the split itself is symmetric/random) but typically far from independent.

Notation you'll need V

Randomized test (recap)

A test may use extra external randomness: an auxiliary random variable U (or V), independent of the data, typically $\text{Unif}[0, 1]$ or a p-variable in its own right. A randomized level- α test rejects according to a rule $\phi(\text{data}, U) \in \{0, 1\}$ with $\mathbb{E}^{\mathbb{P}}[\phi] \leq \alpha$. Randomization lets us build *sharper* (smaller, more powerful) merged p-values than any deterministic rule can achieve, at the cost of an extra coin flip that two people analyzing the same data might not agree on.

Section 12.1 – Intuition: why does the factor 2 appear?

- The most natural way to combine K p-values is to average them: $\bar{P} = \frac{1}{K} \sum_k P_k$. Intuitively small $\bar{P}$ = lots of combined evidence against H_0 .
- But $\bar{P}$ itself is *not* a valid p-value in general! Even for K i.i.d. $\text{Unif}[0, 1]$ p-values, the law of large numbers concentrates $\bar{P}$ tightly around $1/2$ – so $\mathbb{P}(\bar{P} \leq 0.4)$ can be tiny, but $\mathbb{P}(\bar{P} \leq 0.5) \approx 1$, wildly violating $\mathbb{P}(\bar{P} \leq t) \leq t$ for t near $1/2$.
- The fix, remarkably, needs only a factor of exactly 2: *twice* the average of any p-values is always a valid p-value, however they depend on each other.
- **Why 2 and not, say, 1.5 or 3?** This is exactly the same “ $1/\alpha$ ” scaling that appears in **Markov’s inequality** for e-values: a p-to-e calibrator must integrate to 1 over $[0, 1]$, and the specific “triangular” calibrator $f(p) = (2 - 2p)_+$ is the (up to the extremal boundary) canonical choice with $\int_0^1 f = 1$ that is also *linear*, hence compatible with averaging. Chasing this calibrator through Markov’s inequality is precisely how the factor 2 is born – we derive this in full on the next slides.

12.1 The arithmetic mean of p-values I

Proposition 12.1 & Definition 12.2: average p-variables

A random variable P is an *average p-variable* for $\mathbb{P}$ if $\mathbb{E}^{\mathbb{P}}[(v - P)_+] \leq v^2/2$ for all $v \in (0, 1)$.

Equivalently (Proposition 12.1), P is the arithmetic mean of finitely many p-variables, or a (possibly continuous) mixture of p-variables, or satisfies $\mathbb{E}^{\mathbb{P}}[g(P)] \geq \int_0^1 g(u) \, du$ for every increasing concave g .

Plain English: an average p-variable is anything that arises as an average (in a suitably general sense) of genuine p-values. Every p-variable is also an average p-variable (take $K = 1$), but not conversely.

12.1 The arithmetic mean of p-values II

Proposition 12.4: the factor 2, and why it can't be improved

An average p-variable is *not necessarily* a p-variable, but *twice* an average p-variable *always is*. Moreover the factor 2 is sharp: for any $\gamma < 2$, γP fails to be a p-variable for some average p-variable P .

Extremal example: let $U \sim \text{Unif}[0, 1]$. Both U and $1 - U$ are p-variables, so their average, the *constant* $1/2$, is an average p-variable – and no constant smaller than 1 (i.e. no $\gamma/2$ with $\gamma < 2$) can be a valid p-variable, since a constant $c < 1$ has $\mathbb{P}(c \leq t) = 0$ for $t < c$ but $= 1 \geq t$ once $t \geq c$, which already forces $c \geq 1$ for genuine validity at all t ... concretely this construction is why the constant multiplier cannot be pushed below 2.

12.1 The arithmetic mean of p-values III

Theorem 12.6: the calibration bridge between p-values, e-values, and average p-values

For an average p-variable P , $f(P)$ is an e-variable for *any* calibrator f that is *convex* on $[0, 1]$.
Conversely, for any e-variable E , $(2E)^{-1} \wedge 1$ is an average p-variable.

12.1 The arithmetic mean of p-values IV

The three-way calibration relationships

- p-value $\rightarrow$ e-value: $e = f(p)$ for any calibrator f (Ch. 2).
- e-value $\rightarrow$ p-value: $p = e^{-1} \wedge 1$ (the unique admissible choice, Ch. 2).
- average p-value $\rightarrow$ e-value: $e = f(p^*)$, needs f *convex* (Thm. 12.6).
- e-value $\rightarrow$ average p-value: $p^* = (2e)^{-1} \wedge 1$ (Thm. 12.6).
- p-value $\rightarrow$ average p-value: trivial, $p^* = p$ (a p-value is already one).
- average p-value $\rightarrow$ p-value: $p = (2p^*) \wedge 1$ (Prop. 12.4).

Composing the last two rows (average p-value $\rightarrow$ e-value $\rightarrow$ average p-value $\rightarrow$ p-value) recovers exactly the ordinary e-to-p calibration $e \mapsto e^{-1} \wedge 1$ – **average p-values sit exactly halfway between p-values and e-values.**

12.1 The arithmetic mean of p-values V

Theorem 12.8: turning average p-values into level- α tests

Let $V \geq 0$ be independent of P with $\mathbb{E}^{\mathbb{P}}[V \wedge 1] \leq \alpha$. (i) If P is a genuine p-variable, $\mathbf{1}_{\{P \leq V\}}$ is a level- α test. (ii) If P is only an *average* p-variable and V has a decreasing density, $\mathbf{1}_{\{P \leq V\}}$ is *still* a level- α test. In particular, for $U \sim \text{Unif}[0, 1]$ independent of P , $\mathbf{1}_{\{P \leq 2\alpha U\}}$ is a level- α test for $\mathcal{P}$.
Deterministic fallback: without randomization, $\mathbf{1}_{\{(P_1 + \dots + P_K)/K \leq 2\alpha\}}$ is a (weaker but simpler) level- α test – this is exactly “twice the average, compared to α .”

Mid p-values and posterior predictive p-values (12.1.2), briefly

Two important special cases of average p-variables: *mid p-values* $P_T = \frac{1}{2}F(T-) + \frac{1}{2}F(T)$ for a discrete test statistic T (the standard fix for conservative discrete tests), and *posterior predictive p-values* $P_B = \mathbb{P}(D(X', \psi) \geq D(X, \psi) \mid X)$ in Bayesian analysis, obtained by averaging ordinary p-variables over a posterior distribution of an unknown parameter (Proposition 12.1(ii)). Both are average p-variables but generally *not* p-variables themselves – exactly the phenomenon this section is built around.

Step-by-step: how e-values recover “twice the average” (KEY INSIGHT) I

Goal

Show, from scratch, that $2\bar{P} := \frac{2}{K} \sum_{k=1}^K P_k$ is a valid p-value for *arbitrarily dependent* p-variables $P_1, \dots, P_K$ – using only calibration to e-values + Markov’s inequality (no direct probabilistic argument about $\bar{P}$ ’s distribution needed!).

Step 1 – calibrate each P_k to an e-value, at a trial level α

Fix a candidate level $\alpha \in (0, 1)$. The map $p \mapsto (2 - 2p)_+$ is a convex calibrator ($\int_0^1 (2 - 2p) dp = 1$, and it vanishes for $p \geq 1$). By the calibrator-rescaling property (Ch. 2, Prop. 2.6), so is $p \mapsto \gamma(2 - 2\gamma p)_+$ for any $\gamma \geq 1$; hence

$$E_k := \frac{(2 - 2P_k/\alpha)_+}{\alpha} \quad \text{is a genuine e-variable: } \mathbb{E}^{\mathbb{P}}[E_k] \leq 1, \text{ for each } k.$$

Step-by-step: how e-values recover “twice the average” (KEY INSIGHT) II

Step 2 – average the e-values (valid under ANY dependence!)

By linearity of expectation *alone* – no assumption on how $E_1, \dots, E_K$ depend on each other – $\bar{E} := \frac{1}{K} \sum_k E_k$ satisfies $\mathbb{E}^{\mathbb{P}}[\bar{E}] \leq 1$ too. This is the chapter-8 lesson: *averaging e-values is always safe under arbitrary dependence.*

Step-by-step: how e-values recover “twice the average” (KEY INSIGHT) III

Step 3 – apply Markov’s inequality

Since $\bar{E} \geq 0$ and $\mathbb{E}^{\mathbb{P}}[\bar{E}] \leq 1$, Markov’s inequality gives, for every $\alpha \in (0, 1)$,

$$\mathbb{P}\left(\bar{E} \geq \frac{1}{\alpha}\right) \leq \alpha \cdot \mathbb{E}^{\mathbb{P}}[\bar{E}] \leq \alpha.$$

Step-by-step: how e-values recover “twice the average” (KEY INSIGHT) IV

Step 4 – unwind the algebra

$$\bar{E} \geq \frac{1}{\alpha} \iff \frac{1}{K} \sum_k \frac{(2 - 2P_k/\alpha)_+}{\alpha} \geq \frac{1}{\alpha} \iff \frac{1}{K} \sum_k (2 - 2P_k/\alpha)_+ \geq 1.$$

If every $P_k \leq \alpha$ (no clipping by $(\cdot)_+$), this simplifies:

$$\frac{1}{K} \sum_k \left(2 - \frac{2P_k}{\alpha}\right) \geq 1 \iff 2 - \frac{2}{\alpha} \cdot \frac{1}{K} \sum_k P_k \geq 1 \iff \frac{2}{K} \sum_k P_k \leq \alpha \iff 2\bar{P} \leq \alpha.$$

Step-by-step: how e-values recover “twice the average” (KEY INSIGHT) V

Step 5 – conclusion

Combining Steps 3–4: $\mathbb{P}(2\bar{P} \leq \alpha) \leq \mathbb{P}(\bar{E} \geq 1/\alpha) \leq \alpha$ for every $\alpha \in (0, 1)$ (the first inequality can be strict due to clipping, but the bound survives regardless). **This is exactly Proposition 12.4** – recovered purely via calibration + Markov, with no dependence assumption anywhere. The “2” is precisely the constant baked into the calibrator $f(p) = (2 - 2p)_+$, which in turn is forced by $\int_0^1 f = 1$ and f vanishing exactly at $p = 1$.

Running example, part 1: $K = 4$ dependent p-values, the direct route

Setup

Four (arbitrarily dependent – e.g. from overlapping/correlated sub-studies) p-values:

$$p_1 = 0.02, \quad p_2 = 0.15, \quad p_3 = 0.30, \quad p_4 = 0.60.$$

Classical route: just average and double

$$\bar{p} = \frac{0.02 + 0.15 + 0.30 + 0.60}{4} = \frac{1.07}{4} = 0.2675, \quad 2\bar{p} = 0.535.$$

By Proposition 12.4, 0.535 **is a valid merged p-value**, regardless of how $P_1, \dots, P_4$ depend on each other – e.g. even if they arose from four highly overlapping analyses of the same data.

Sanity check against a naive competitor

Bonferroni ($K \min(p_k) = 4 \times 0.02 = 0.08$) is also valid but far more conservative here since one very small p-value gets diluted by three large ones; the averaging rule rewards *several moderately small* p-values, Bonferroni rewards a *single very small* one.

Running example, part 1 continued: the e-value route lands in the same place I

Step 1: calibrate each p_k to an e-value at trial level $\alpha = 0.535$

Using $E_k = (2 - 2p_k/\alpha)_+/\alpha$ with $\alpha = 0.535$:

$$E_1 = 3.599, \quad E_2 = 2.690, \quad E_3 = 1.642, \quad E_4 = 0 \quad (\text{since } p_4/\alpha = 1.1215 > 1, \text{ clipped}).$$

Step 2: average, then compare to $1/\alpha$

$$\bar{E} = \frac{3.599 + 2.690 + 1.642 + 0}{4} = 1.983, \quad \frac{1}{\alpha} = \frac{1}{0.535} = 1.869.$$

Indeed $\bar{E} = 1.983 \geq 1.869 = 1/\alpha$: the event that certifies validity of $\alpha = 0.535$ (Step 4–5 of the derivation) actually holds for these numbers – **the e-value route and the direct route certify the very same merged p-value, 0.535.**

Running example, part 1 continued: the e-value route lands in the same place II

Bonus: a sharper (admissible) calibrator does slightly better

Solving the general merging formula (Section 12.2, Example 12.21) with the *same* calibrator $f(p) = (2 - 2p)_+$ used directly (not just as a validity check) gives the tighter merged p-value $F(\mathbf{p}) = 0.47 < 0.535$ – see the Python cell below.

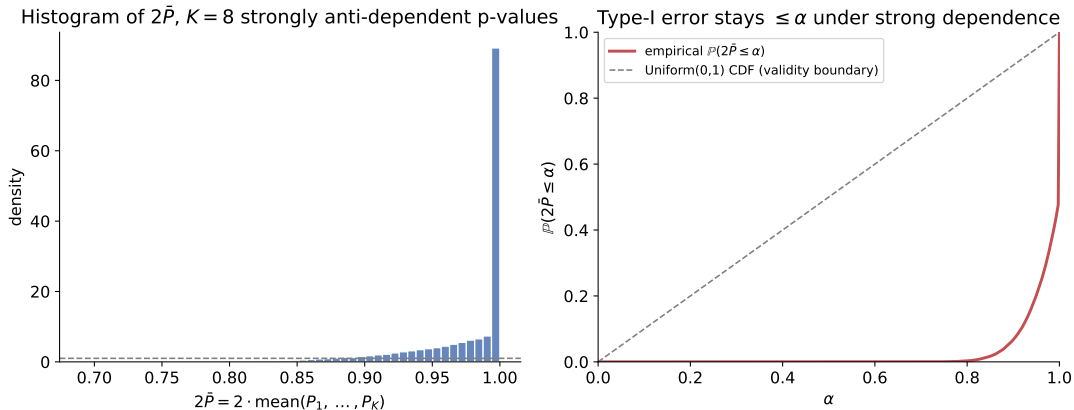
Python: 12.1 – the e-value route, numerically

```
1 import numpy as np
2
3 def f_calibrator(x):
4     return np.clip(2.0 - 2.0*x, 0.0, None)           # convex calibrator  $(2-2p)_+$ 
5
6 p = np.array([0.02, 0.15, 0.30, 0.60])
7 alpha = 2 * p.mean()
8 print(f"2*mean = {alpha:.3f}")                      # 0.535
9
10 E = f_calibrator(p/alpha) / alpha                  # e-values at trial level alpha
11 print("e-values:", np.round(E, 3))
12 print("mean(E) =", round(E.mean(), 3), " 1/alpha =", round(1/alpha, 3))
13 # mean(E) >= 1/alpha -> confirms 0.535 is a valid merged p-value
```

Python: 12.1 – validating $2\bar{P}$ under strong dependence

```
1 import numpy as np
2 from scipy.stats import norm
3 rng = np.random.default_rng(2024)
4
5 # Strong-dependence simulation: K=8 p-values, two anti-dependent halves
6 K, N, rho = 8, 50000, 0.95
7 W = rng.standard_normal(N)
8 eps = rng.standard_normal((N, K))
9 loadings = np.where(np.arange(K) < K//2, rho, -rho) # half +rho, half -rho
10 Z = loadings * W[:, None] + np.sqrt(1 - rho**2) * eps
11 P = norm.cdf(Z) # each column ~ Unif(0,1)
12 twice_mean = np.minimum(2 * P.mean(axis=1), 1.0)
13
14 for a in [0.05, 0.2, 0.5]:
15     emp = (twice_mean <= a).mean()
16     print(f"alpha={a:.2f}: empirical P(2*mean<=alpha)={emp:.4f} (must be <= {a})")
```

Figure: $2\bar{P}$ stays valid even under strong (constructed) dependence



Left: histogram of $2\bar{P}$ for $K = 8$ p-values built to be strongly *anti*-dependent across two halves (a Gaussian-copula construction, loadings ± 0.95 on a common factor) – the same qualitative mechanism as the extremal example $P_1 = U$, $P_2 = 1 - U$ behind Proposition 12.4. Right: the empirical CDF of $2\bar{P}$ never crosses above the Uniform(0, 1) diagonal – Type-I error control holds at every level α simultaneously, exactly as the Markov-inequality derivation guarantees.

Section 12.2 – Intuition: why go through e-values for *general* merging?

- Section 12.1 handled one specific merging rule (the mean). What about *any* sensible way of combining K p-values – weighted means, order statistics, geometric means, Simes' rule, ...?
- The key structural fact (via an optimal-transport duality argument, Lemma 11.1) is that constraints on *probabilities* ($\mathbb{P}(\text{reject}) \leq \varepsilon$) are dual to constraints on *expectations* ($\mathbb{E}[\cdot] \leq 1$). E-values live natively in expectation-land; p-values live in probability-land. Converting is exactly what lets us use Markov's inequality, which is an expectation-to-probability bridge.
- The remarkable payoff (Theorem 12.15 below): *every single* admissible way of merging p-values under arbitrary dependence – no matter how exotic – can be written as “calibrate each p_k to an e-value, take a weighted average, apply Markov.” There is no other game in town.

12.2.1 P-merging functions and examples I

Rejection regions – an equivalent way to think about F

The *rejection region* of a p-merging function F at level ε is $R_\varepsilon(F) = \{\mathbf{p} : F(\mathbf{p}) \leq \varepsilon\}$. For homogeneous F , $R_\varepsilon(F) = \varepsilon A$ for a fixed set A ; conversely, any nested increasing family of Borel lower sets $\{R_\varepsilon\}_{\varepsilon \in (0,1)}$ defines $F(\mathbf{p}) = \inf\{\varepsilon : \mathbf{p} \in R_\varepsilon\}$. Validity of F is equivalent to $\mathbb{P}(\mathbf{p} \in R_\varepsilon) \leq \varepsilon$ for all ε – this reformulation is what makes the Markov-inequality machinery bite.

Two natural families (Examples 12.11–12.13)

Order-family: $G_{k,K}(\mathbf{p}) = \frac{K}{k} p_{(k)} \wedge 1$ using the k -th order statistic; $k = 1$ gives **Bonferroni** ($K \min_k p_k$), $k = K$ gives the (invalid on its own, but instructive) **maximum** $\max_k p_k$.

Mean-family: $F_{r,K}(\mathbf{p}) = b_{r,K} \mathbb{M}_{r,K}(\mathbf{p}) \wedge 1$ where $\mathbb{M}_{r,K}$ is the generalized (power) mean of order r ; $r = 1$ is the arithmetic mean of Section 12.1, $r \rightarrow 0$ the geometric mean, $r \rightarrow -\infty$ the min (Bonferroni again), $r \rightarrow \infty$ the max.

12.2.1 P-merging functions and examples II

Simes and Hommel functions (Example 12.14)

Minimizing the order-family over k gives the *Simes function* $S_K = \bigwedge_k G_{k,K}$ – valid only under independence or PRDS, *not* arbitrary dependence, but it is the smallest possible *symmetric* p-merging function pointwise. Penalizing by $\ell_K = \sum_{k=1}^K 1/k$ gives the *Hommel function* $H_K = \ell_K S_K$, which *is* valid under arbitrary dependence.

12.2.1 P-merging functions and examples III

Definition (12.2.2): merging p-values by merging e-values

Fix calibrators $f_1, \dots, f_K$ and weights $(\lambda_1, \dots, \lambda_K)$ in the simplex $\Delta_K = \{\lambda \in [0, 1]^K : \sum_k \lambda_k = 1\}$. Define

$$G(\mathbf{p}) := \lambda_1 f_1(p_1) + \dots + \lambda_K f_K(p_K).$$

For any p-variables $\mathbf{p} = (P_1, \dots, P_K)$, $G(\mathbf{p})$ is a *convex combination of e-variables*, hence itself an e-variable: $\mathbb{E}^{\mathbb{P}}[G(\mathbf{p})] \leq 1$ – true for *any* joint dependence among the P_k (again, Ch. 8's lesson). Markov's inequality then gives $\mathbb{P}(G(\mathbf{p}) \geq 1/\varepsilon) \leq \varepsilon$, so $1/G$ is a valid (if typically inadmissible/wasteful) p-merging function.

12.2.1 P-merging functions and examples IV

Theorem 12.15 & 12.17: this construction is not just *a* way – it's *the* way

(12.15) Every admissible p-merging function's rejection region can be represented, level by level, as $R_\varepsilon(F) = \{\mathbf{p} : \sum_k \lambda_k f_k(p_k/\varepsilon) \geq 1\}$ for some weights and calibrators (possibly depending on ε , but generated by one fixed tuple via rescaling). (12.17) If we also demand F be *symmetric*, a single shared calibrator f suffices:

$$F(\mathbf{p}) = \inf \left\{ \varepsilon \in (0, 1] : \frac{1}{K} \sum_{k=1}^K f\left(\frac{p_k}{\varepsilon}\right) \geq 1 \right\}.$$

12.2.1 P-merging functions and examples V

Examples 12.20–12.24: named merging rules, and their calibrators

Rule	Calibrator $f(p)$	Note
Order-family $G_{k,K}$	$\frac{K}{k} \mathbf{1}_{\{p \in [0, k/K]\}}$	admissible for $k \leq K-1$
2× arithmetic mean	$(2 - 2p)_+$	Sec. 12.1; slight improvement possible
e× geometric mean	$(-\log p)_+$	only approx. precise, large K
$(\log K + \log \log K + 1) \times$ harmonic mean	$(1/(T_K p) - 1/T_K) \wedge K$	approx. precise, large K
Hommel H_K	piecewise, see Ex. 12.24	admissible iff K not prime

Theorem 12.18 gives a checkable sufficient condition for admissibility: a strictly convex calibrator with $f \leq K$ on $(0, 1]$ and $f(1) = 0$ always induces an admissible p-merging function.

Running example, part 2: the general merging formula, worked out

Setup: same $K = 4$ numbers, calibrator $f(p) = (2 - 2p)_+$, symmetric weights

Using Theorem 12.17's formula with $f(p) = (2 - 2p)_+$: solve for the smallest ε with $\frac{1}{4} \sum_k (2 - 2p_k/\varepsilon)_+ \geq 1$.

Solving by hand

Try $\varepsilon \in [0.30, 0.60)$: then $p_1, p_2, p_3 \leq \varepsilon$ (unclipped) but $p_4 = 0.60 > \varepsilon$ (clipped to 0):

$$\frac{1}{4} \left[\left(2 - \frac{2(0.02)}{\varepsilon}\right) + \left(2 - \frac{2(0.15)}{\varepsilon}\right) + \left(2 - \frac{2(0.30)}{\varepsilon}\right) \right] = 1 \implies 6 - \frac{2(0.47)}{\varepsilon} = 4 \implies \varepsilon = 0.47.$$

Since $0.47 \in [0.30, 0.60)$, this is consistent: **the tightest merged p-value from this calibrator is $F(\mathbf{p}) = 0.47$** – strictly smaller (more powerful) than the plain $2\bar{p} = 0.535$ from Section 12.1, because clipping the fourth (large, uninformative) p-value out of the sum lets the other three carry more weight.

Lesson

“2×mean” is the simplest valid rule, but not the sharpest one built from the very same calibrator – admissibility (Theorem 12.18) squeezes out this last bit of conservativeness.

Python: 12.2 – general p-merging via bisection on ε

```
1  import numpy as np
2
3  def f_calibrator(x):
4      return np.clip(2.0 - 2.0*x, 0.0, None)          # same calibrator as before
5
6  def merged_pvalue(p_row):
7      """Smallest eps solving (1/K) sum_k f(p_k/eps) >= 1, i.e. Theorem 12.17."""
8      K = len(p_row)
9      def criterion(eps):
10         return np.mean(f_calibrator(p_row/eps))
11     lo, hi = 1e-12, 1.0
12     while criterion(hi) < 1.0 and hi < 1e6:
13         hi *= 2.0                                     # criterion is increasing in eps
14         for _ in range(60):                             # bisection
15             mid = 0.5*(lo+hi)
16             if criterion(mid) >= 1.0:
17                 hi = mid
18             else:
19                 lo = mid
20     return hi
21
22 p = np.array([0.02, 0.15, 0.30, 0.60])
23 print("classical 2*mean          :", round(2*p.mean(), 3))          # 0.535
24 print("admissible merged p-value:", round(merged_pvalue(p), 3))    # 0.47  -- sharper!
```

Both are valid p-values here; 0.47 is the sharper, admissible one.

Section 12.3 – Intuition: what changes under exchangeability?

- Recall: $(P_1, \dots, P_K)$ *exchangeable* means permuting them doesn't change their joint law – like recording cards drawn from a shuffled deck, order-dependent labels carry no extra statistical information.
- You might guess exchangeability should make *symmetric* merging rules (like the ones in 12.2) even better. **Surprisingly, it does not:** Proposition 12.26 shows any *symmetric* rule that is valid under exchangeability is automatically valid under arbitrary dependence too – exchangeability buys nothing extra if you insist on symmetry.
- The real gain comes from deliberately *breaking* symmetry: process the p-values *in the order they arrive* and look at running (prefix) averages. Exchangeability guarantees this is still safe, precisely because any arrival order is statistically as good as any other – so “getting lucky” with an early small p-value is not cheating.

12.3 Combining exchangeable p-values I

Definition 12.25: ex-p-merging function

F is an *ex-p-merging function* if $\mathbb{P}(F(\mathbf{p}) \leq \alpha) \leq \alpha$ for all $\alpha \in (0, 1)$ whenever $\mathbf{p}$ is *exchangeable* (rather than for every dependence structure). Since exchangeability is a weaker restriction on the input than “arbitrary,” more functions qualify – *a priori* more room for improvement.

Proposition 12.26: symmetric functions gain nothing

Any *symmetric* ex-p-merging function is automatically an ordinary p-merging function (valid under arbitrary dependence too). *Proof idea:* given arbitrary $\mathbf{p}$, randomly permute it with an independent uniform permutation π ; the permuted vector $\mathbf{p}^\pi$ is exchangeable by construction, so $F(\mathbf{p}^\pi)$ is a valid p-value – but symmetry means $F(\mathbf{p}^\pi) = F(\mathbf{p})$ always, so $F(\mathbf{p})$ was valid all along.

12.3 Combining exchangeable p-values II

Theorem 12.27: the actual improvement – prefix averages, taken online

For any calibrator f , define

$$F(\mathbf{p}) = \inf \left\{ \varepsilon \in (0, 1] : \bigvee_{\ell=1}^K \left(\frac{1}{\ell} \sum_{k=1}^{\ell} f\left(\frac{p_k}{\varepsilon}\right) \right) \geq 1 \right\}.$$

Then $F(\mathbf{p})$ is a valid p-value *whenever $\mathbf{p}$ is exchangeable* – note F is *not* symmetric in general (it depends on the *order* $p_1, \dots, p_K$), consistent with Proposition 12.26. Since we maximize over *all* prefixes $\ell = 1, \dots, K$, this F is always $\leq$ the single-average rule of Theorem 12.17 (which only checks $\ell = K$) – **strictly more powerful, for free**, whenever exchangeability holds.

12.3 Combining exchangeable p-values III

Bonus: works online, without fixing K in advance

Because the criterion only ever needs the prefix $p_1, \dots, p_\ell$, p-values can be processed *one at a time as they arrive* and the procedure can be stopped at any point to get a valid merged p-value so far – provided the calibrator f itself does not depend on K . This is exactly the appeal of processing repeated sample-splits of the same dataset: each new split gives one more exchangeable p-value, and Theorem 12.27 lets you keep monitoring validly.

Running example, part 3: exploiting exchangeability

Same numbers, now assumed exchangeable

$p_1 = 0.02, p_2 = 0.15, p_3 = 0.30, p_4 = 0.60$ (imagine these arrived in this order from repeated random sample splits).

Prefix $\ell = 1$ already does all the work

With $f(p) = (2 - 2p)_+$, the $\ell = 1$ term alone requires $2 - 2p_1/\varepsilon \geq 1 \iff \varepsilon \geq 2p_1 = 0.04$. Checking the max over all prefixes $\ell = 1, \dots, 4$ (the $\ell = 4$ term alone would give $\varepsilon = 0.47$ as before), the *smallest* valid ε is driven by $\ell = 1$:

$$F(\mathbf{p}) = 0.04.$$

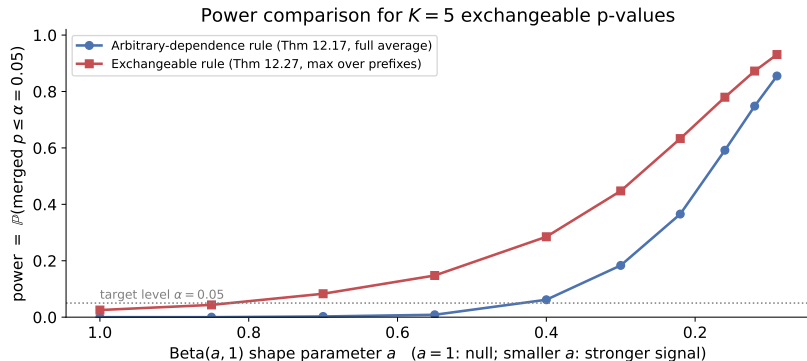
Why this is legitimate

Under *arbitrary* dependence, singling out “whichever p_k happens to be smallest” and treating it alone at level $2p_{(1)}$ would *not* be valid in general (you’d be data-snooping the smallest of K values without correction). Under *exchangeability*, though, the position in which a small p-value appears carries no information – so this aggressive, order-exploiting rule remains exactly level- α . This is a genuine, free improvement from 0.47 down to 0.04.

Python: 12.3 – prefix-average merge for exchangeable p-values

```
1 import numpy as np
2
3 def f_calibrator(x):
4     return np.clip(2.0 - 2.0*x, 0.0, None)
5
6 # use_max_prefix=False: Thm 12.17 (arbitrary dependence, full average).
7 # use_max_prefix=True : Thm 12.27 (exchangeable, max over running prefixes).
8 def merged_pvalue(p_row, use_max_prefix):
9     K = len(p_row)
10    def criterion(eps):
11        vals = f_calibrator(p_row/eps)
12        if use_max_prefix:
13            cums = np.cumsum(vals)
14            return np.max(cums / np.arange(1, K+1)) # prefix average, maxed
15        return np.mean(vals)
16    lo, hi = 1e-12, 1.0
17    while criterion(hi) < 1.0 and hi < 1e6:
18        hi *= 2.0
19    for _ in range(60): # bisection for smallest eps
20        mid = 0.5*(lo+hi)
21        lo, hi = (lo, mid) if criterion(mid) >= 1.0 else (mid, hi)
22    return hi
23
24 p = np.array([0.02, 0.15, 0.30, 0.60])
25 print("arbitrary-dependence (Thm 12.17):", round(merged_pvalue(p, False), 3)) # 0.47
26 print("exchangeable (Thm 12.27):", round(merged_pvalue(p, True), 3)) # 0.04
```

Figure: power gain from exploiting exchangeability



$K = 5$ exchangeable p-values simulated as i.i.d. Beta($a, 1$) (randomly reordered each replicate to make exchangeability, not independence, the only thing used); $a = 1$ is the null, smaller a means more real signal. At every signal strength, the prefix-average rule (Theorem 12.27) rejects more often than the full-average rule (Theorem 12.17) at the same nominal level $\alpha = 0.05$, while both control Type-I error at $a = 1$.

Section 12.4 – Intuition: what does randomization buy us?

- Sections 12.1–12.3 all end in a Markov-inequality step: $\mathbb{P}(\bar{E} \geq 1/\varepsilon) \leq \varepsilon \cdot \mathbb{E}[\bar{E}] \leq \varepsilon$. Plain Markov compares $\bar{E}$ to the *fixed* threshold $1/\varepsilon$.
- Chapter 2's *randomized* Markov inequality replaces this fixed threshold with a threshold that itself depends on an independent auxiliary random variable – this squeezes extra power out of exactly the same e-values, at the cost of injecting outside randomness that two analysts of the same data might resolve differently.
- This is the same idea already glimpsed in Theorem 12.8 ($\mathbf{1}_{\{P \leq 2\alpha U\}}$ is a sharper, randomized version of $\mathbf{1}_{\{2\bar{P} \leq \alpha\}}$); Theorem 12.28 extends it to the fully general merging-function setting of Section 12.2.

12.4 Randomized combinations of p-values I

Theorem 12.28: randomized combination

For weights $(\lambda_1, \dots, \lambda_K) \in \Delta_K$ and calibrators $f_1, \dots, f_K$, define

$$F(\mathbf{p}, u) = \inf \left\{ \varepsilon \in (0, 1] : \sum_{k=1}^K \lambda_k f_k \left(\frac{p_k}{\varepsilon} \right) \geq u \right\}.$$

Then for any p-variables $\mathbf{p}$ and any p-variable V *independent* of $\mathbf{p}$, $F(\mathbf{p}, V)$ is itself a valid p-value. Validity for $U \sim \text{Unif}[0, 1]$ follows from the randomized Markov inequality (Ch. 2) applied to the e-variable $\sum_k \lambda_k f_k(P_k/\varepsilon)/\varepsilon$ in place of plain Markov's inequality; validity for a general independent p-variable V then follows since $F(\mathbf{p}, u)$ is increasing in u , so $\mathbb{P}(F(\mathbf{p}, V) \leq \alpha) \leq \mathbb{P}(F(\mathbf{p}, U) \leq \alpha) \leq \alpha$.

12.4 Randomized combinations of p-values II

Practical remedy when randomization is undesirable

If one of the inputs, say P_k , is known to be independent of the rest, you can set $V = P_k$ itself (no extra coin flip needed) and merge the remaining $K - 1$ p-values with randomization “for free.” Otherwise, if randomization is permitted at all, apply the exchangeable rule of Section 12.3 to a *randomly permuted* ordering of the arbitrarily-dependent p-values – turning Section 12.2’s problem into Section 12.3’s easier one.

12.4 Randomized combinations of p-values III

Table 12.29 – summary of all combination rules in this chapter

	Arbitrary dep. (12.2)	Exchangeable (12.3)	Arbitrary, randomized (12.4)
Order stats	$\frac{K}{k} p_{(k)}$	$\frac{K}{k} \bigwedge_m p_{(\lambda_m)}^m$	$\frac{K}{k} p_{(\lceil Uk \rceil)}$
Arithmetic mean	$2 \mathbb{M}_1(\mathbf{p})$	$2 \bigwedge_m \mathbb{M}_1(\mathbf{p}_m)$	$\frac{2}{2-U} \mathbb{M}_1(\mathbf{p})$
Geometric mean	$e \mathbb{M}_0(\mathbf{p})$	$e \bigwedge_m \mathbb{M}_0(\mathbf{p}_m)$	$e^U \mathbb{M}_0(\mathbf{p})$
Harmonic mean	$(T_K+1) \mathbb{M}_{-1}(\mathbf{p})$	$(T_K+1) \bigwedge_m \mathbb{M}_{-1}(\mathbf{p}_m)$	$(T_K U+1) \mathbb{M}_{-1}(\mathbf{p})$

Here $U \sim \text{Unif}[0, 1]$ independent, $T_K = \log K + \log \log K + 1$, and $\mathbf{p}_m$ denotes the first m p-values – randomization (rightmost column) improves the arbitrary-dependence rules using the very same machinery as the exchangeable rules, but without needing an exchangeability assumption.

Running example, part 4: what randomization buys, in numbers

Same $K = 4$ numbers, now allowing an independent draw $U \sim \text{Unif}[0, 1]$

Using the randomized version of Theorem 12.28 with $f(p) = (2 - 2p)_+$, solve $\frac{1}{4} \sum_k f(p_k/\varepsilon) \geq u$ for the drawn value of u :

$u = 1.0$ (no randomization, back to Thm 12.17) :	$F(\mathbf{p}, 1) = 0.47,$
$u = 0.5$:	$F(\mathbf{p}, 0.5) = 0.17,$
$u = 0.3$:	$F(\mathbf{p}, 0.3) = 0.05,$
$u = 0.1$:	$F(\mathbf{p}, 0.1) = 0.025.$

Interpretation

A *smaller* realized draw u (which happens with probability u under $U \sim \text{Unif}[0, 1]$) gives a *smaller*, more powerful merged p-value – but validity is only guaranteed on *average over the randomness of U* : the randomized Markov inequality ensures $\mathbb{P}(F(\mathbf{p}, U) \leq \alpha) \leq \alpha$ still holds after integrating out U , even though any single realized u can look either lucky or unlucky.

Chapter 12 summary

- **12.1:** the arithmetic mean of p-values is not itself a p-value, but *twice* the mean always is (Prop. 12.4) – and this “2” is exactly the constant forced by a convex p-to-e calibrator $f(p) = (2 - 2p)_+$ combined with Markov’s inequality (Thm. 12.6). Average p-values sit strictly between p-values and e-values in the calibration hierarchy.
- **12.2:** every admissible way of merging p-values under arbitrary dependence secretly factors through merging e-values via calibrators and weights, then applying Markov’s inequality (Thm. 12.15, 12.17) – this is not one trick among many, it is the *only* trick.
- **12.3:** exchangeability alone does not help *symmetric* rules (Prop. 12.26) – but deliberately breaking symmetry via running/prefix averages (Thm. 12.27), justified precisely because exchangeable data doesn’t care about order, gives a strict, free improvement.
- **12.4:** randomization upgrades Markov’s inequality to a randomized version (Thm. 12.28), buying further power from arbitrarily dependent p-values without any distributional assumption, at the cost of extra external randomness.
- **Running numeral:** $p = (0.02, 0.15, 0.30, 0.60)$ merges to 0.535 (classical $2 \times \text{mean}$), 0.47 (admissible arbitrary-dependence rule), 0.04 (exploiting exchangeability), or as low as 0.025–0.05 (with randomization) – the *same* four numbers, four increasingly sharp but always-valid answers, depending on what you are willing to assume or randomize.